

ApexATM: ATM Simulation System

Project Report – VITyarthi Build Your Own Project

Field	Details
Student Name	Aditya Raj
Registration Number	25BAI10708
Programme	B.Tech CSE (AI/ML), 2nd Year
Course	CSE2006
Project Title	ApexATM – Java ATM Simulator
GitHub Repository	https://github.com/adityarghh/ApexATM-Vityarthi.git

1. Introduction

ApexATM is a desktop-based ATM terminal simulation built in Java using the Swing toolkit. I built this project to apply core Object-Oriented Programming (OOP) concepts—such as class design, encapsulation, and modular layered architecture—inside a practical desktop application rather than running plain terminal scripts.

The program allows a user to sign in with their User ID and 4-digit PIN. Once authenticated, they can perform standard everyday banking operations: check account balances, withdraw money, make deposits, execute quick cash transactions, transfer money, pay bills, change their security PIN, and review a mini statement of recent activity.

2. Problem Statement

Real-world ATM machines handle financial transactions where accuracy is non-negotiable. An ATM cannot afford unhandled crashes when users enter letters instead of numbers, and it must strictly prevent overdrafts.

In beginner-level GUI assignments, developers often make the mistake of dumping interface buttons, account balances, and banking rules into one single file. This creates messy, brittle code where fixing a button can accidentally break how money is calculated.

ApexATM solves this by separating the business logic (services) from what the user sees (UI panels), ensuring all numbers are checked and validated before balances change.

2.1 Scope

- Verify user credentials via User ID and masked 4-digit PIN.
- Handle core operations: balance checks, cash withdrawals, deposits, quick cash, transfers, and bill payments.
- Track transaction history in memory during the active session.
- Validate user inputs to prevent negative amounts, overdrafts, and invalid characters.

2.2 Target Users

- Banking customers looking for a clean, simple self-service kiosk.
- Course evaluators reviewing Java OOP design, clean structure, and desktop GUI implementations.

3. Functional Requirements

The application is divided into three functional modules:

Module	Core Purpose
Authentication Module	Verifies login credentials (User ID and 4-digit PIN). Invalid combinations are caught and display an error message.
Account Operations Module	Lets users view current balances, withdraw cash, take a quick \$50 cash withdrawal, deposit funds, transfer money, pay bills, and update their PIN.
Transaction History Module	Automatically logs deposits and withdrawals with action type, amount, post-transaction balance, and timestamp, rendering them in a mini statement.

Every operation follows a straightforward Input → Process → Output flow: the user enters an amount or detail into a prompt, the underlying service class checks the logic, and the user interface reflects the updated balance or displays an error.

4. Non-Functional Requirements

Property	How It Is Handled
Security	PIN inputs use JPasswordField masking to prevent shoulder-surfing. Users cannot access account screens without a valid ID/PIN match.
Reliability	Overdraft checks run before any money leaves the account. If a user tries to withdraw more than they have, the transaction halts immediately.
Usability	Uses a modern layout with clean typography, obvious button labels, and dialog prompts so the workflow is easy to follow.
Error Handling	Text inputs are parsed inside try-catch blocks. Empty spaces, letters, and invalid PIN formats are caught without crashing the application.
Maintainability	Divided cleanly into atm.model, atm.service, and atm.ui packages. The UI can be adjusted without risking the math logic.
Performance	Operations run directly in memory using standard Java collections, completing transactions instantly with no network delay.

5. System Architecture

ApexATM uses a 3-tier layered architecture:

- UI Layer: LoginPanel and DashboardPanel handle mouse clicks, user inputs, and screen drawing.
- Service Layer: AuthService handles authentication; AccountService handles balance updates and banking limits.
- Model Layer: User and Transaction store account details, balances, and history logs.

- Main: The entry file that sets up the services and coordinates screen transitions.

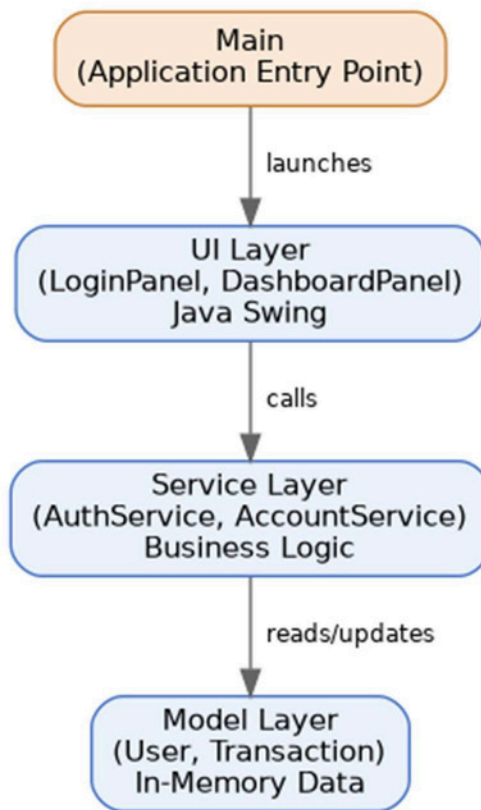


Figure 1: Layered Architecture Diagram

6. Design Diagrams

6.1 Use Case Diagram

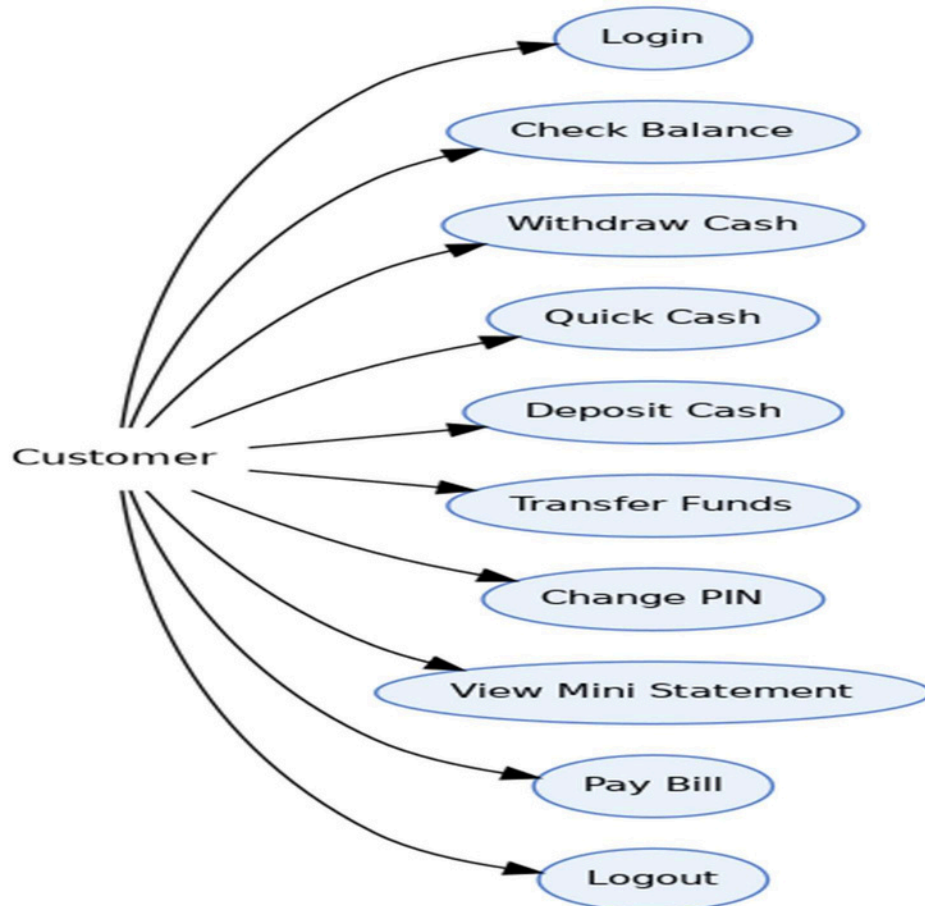


Figure 2: Use Case Diagram

6.2 Class Diagram

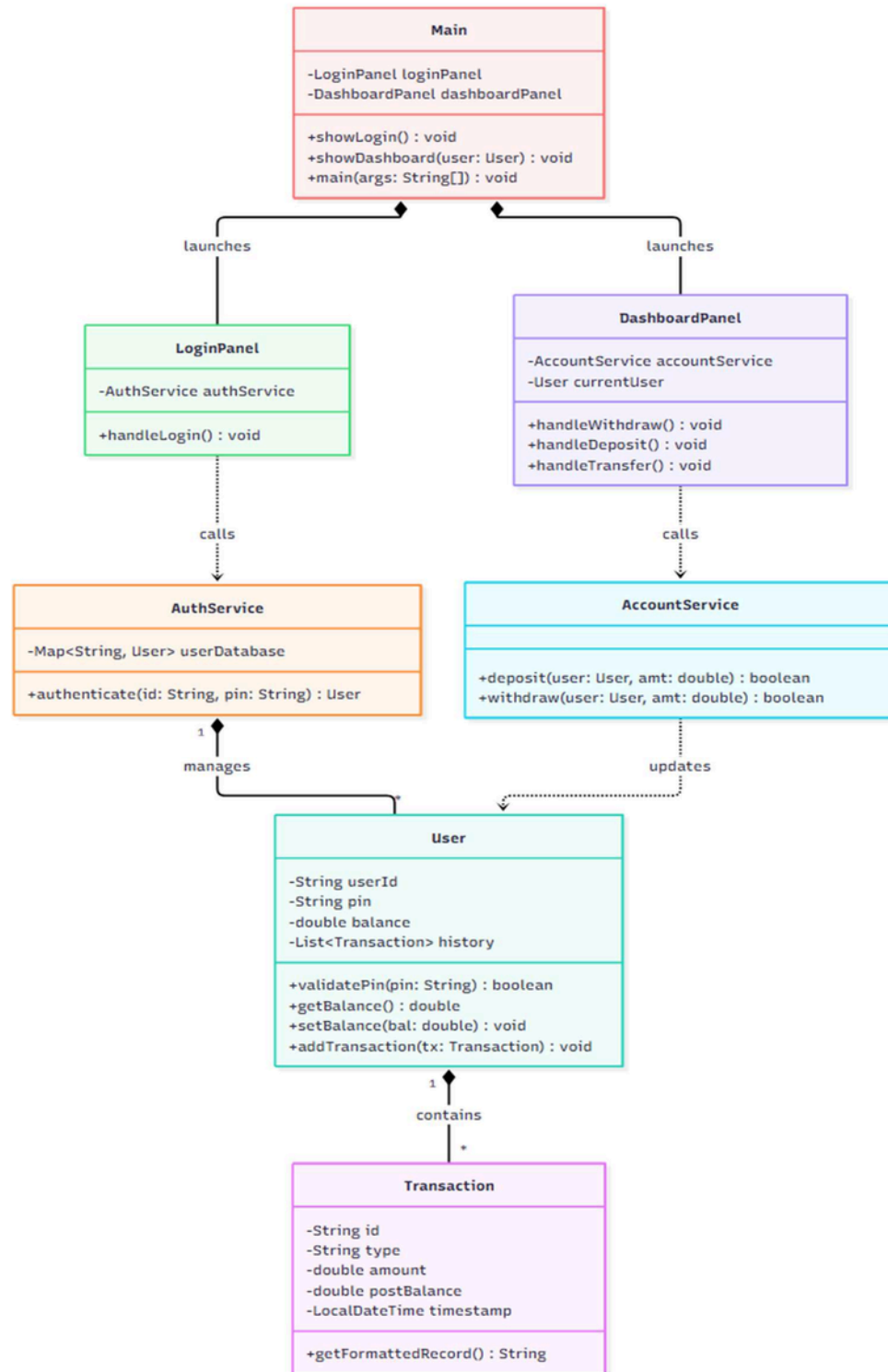


Figure 3: Class Diagram

6.3 Sequence Diagram (Login & Withdrawal)

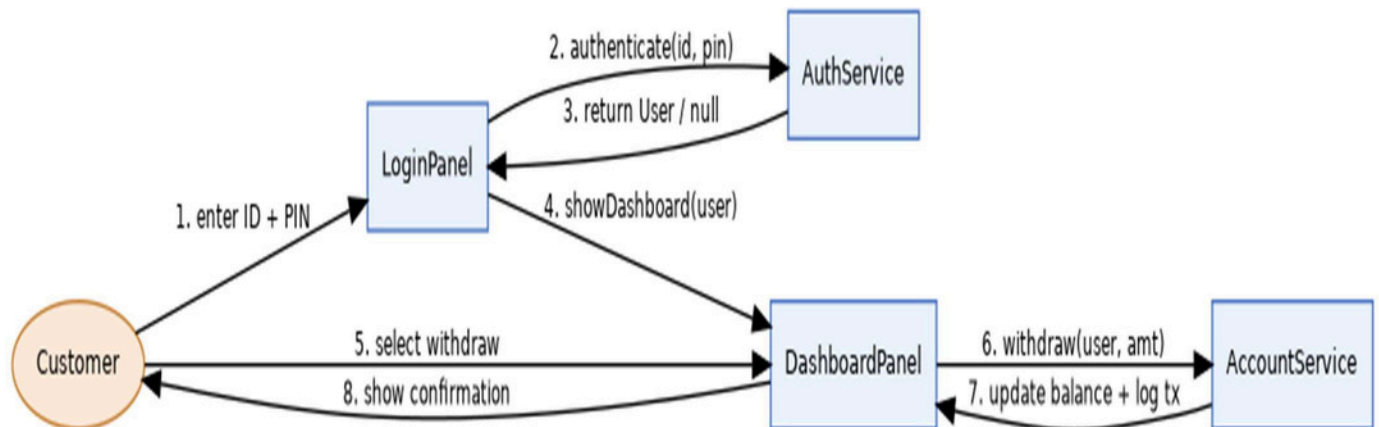


Figure 4: Sequence Diagram

7. Design Decisions & Rationale

- Swing instead of a web app: Swing was chosen because an ATM is fundamentally a dedicated standalone physical terminal. Building it as a desktop GUI keeps the setup simple and evaluation hassle-free with zero server setup.
- Layered packages (model, service, ui): Isolating calculations in AccountService allows modifications to visual buttons without breaking balance math.
- In-memory data structures: Using an internal HashMap was preferred over a heavy SQL database. It meets project goals without requiring manual database installations on the evaluator's system.
- Single window using CardLayout: Swapping views inside one JFrame provides a cleaner user experience than popping open new windows for every action.
- Service-level checks: Defensive rules (like rejecting withdrawals greater than the balance) are enforced inside the service classes, so bad requests are caught even if UI input checks fail.

8. Implementation Details

File Path	What It Handles
src/atm/Main.java	Starts the app, initializes services, and swaps screens using CardLayout.
src/atm/model/User.java	Holds user data: ID, PIN, balance, and transaction history.
src/atm/model/Transaction.java	Stores individual transaction records with timestamps and amounts.
src/atm/service/AuthService.java	Stores users in memory and checks login credentials.
src/atm/service/AccountService.java	Handles financial operations (deposits, withdrawals, transfers) and enforces balance limits.
src/atm/ui/LoginPanel.java	Login screen collecting User ID and PIN with custom card graphics.
src/atm/ui/DashboardPanel.java	Main banking terminal screen displaying balance readouts and transaction buttons.

Demo Test Accounts:

- User ID: user123 | PIN: 1234 | Starting Balance: \$1500.00
- User ID: admin | PIN: 9999 | Starting Balance: \$5000.00

9. Screenshots / Results

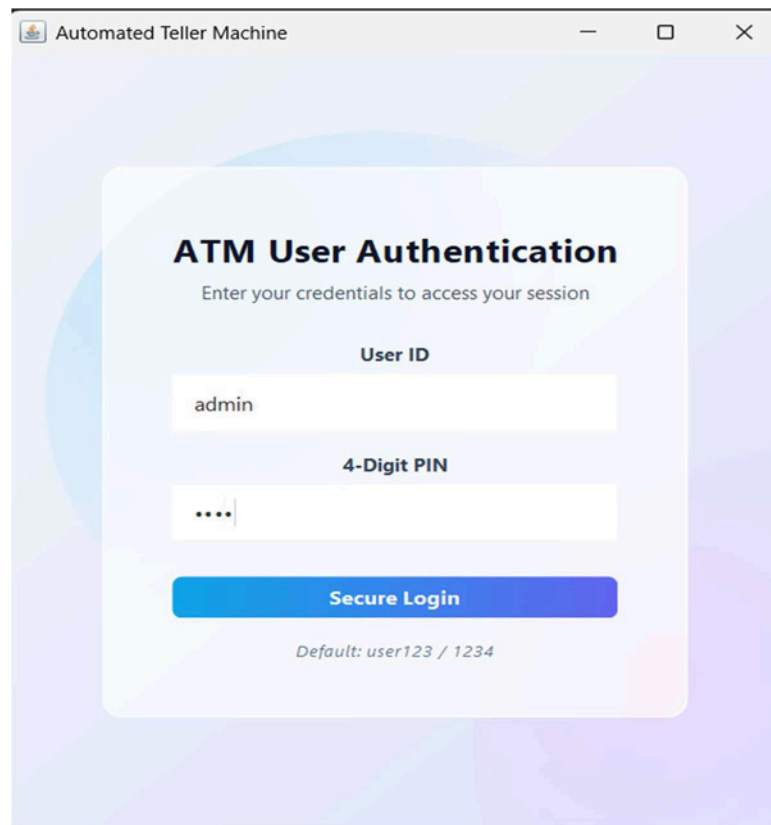


Figure 5: Login Screen — Sign-in card with User ID and masked PIN text fields

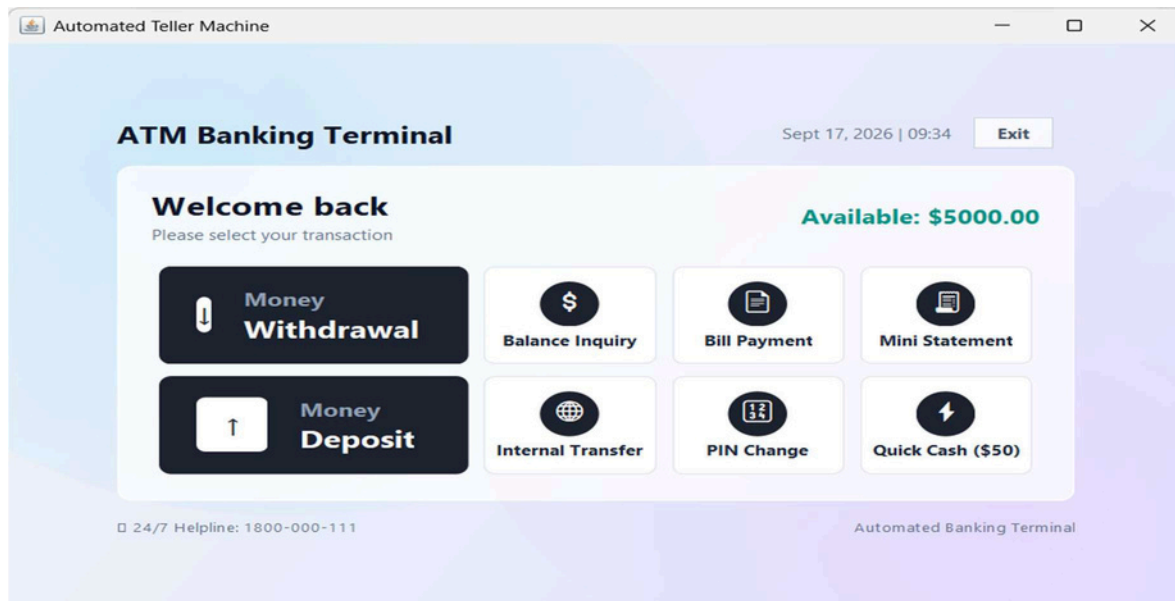


Figure 6: ATM Dashboard Terminal — Main screen displaying balance and transaction buttons

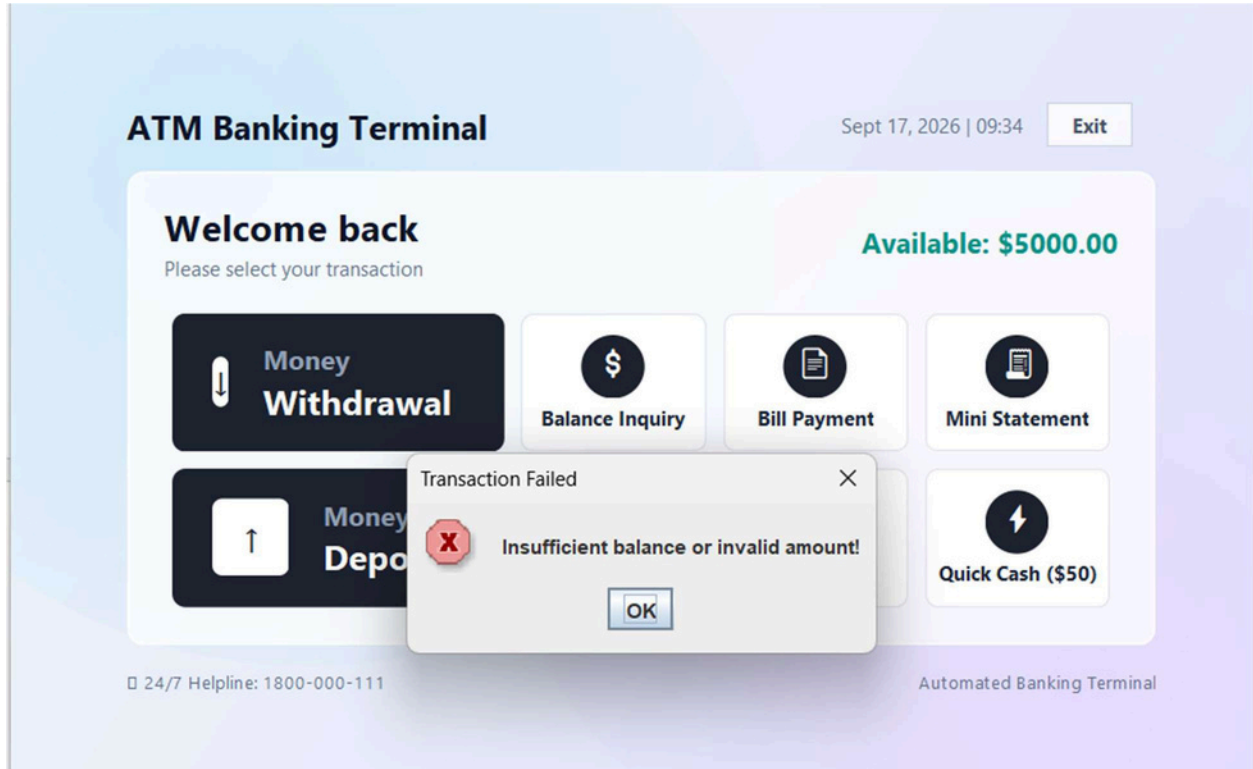


Figure 7: Input Prompt & Overdraft Handling — Overdraft warning dialog

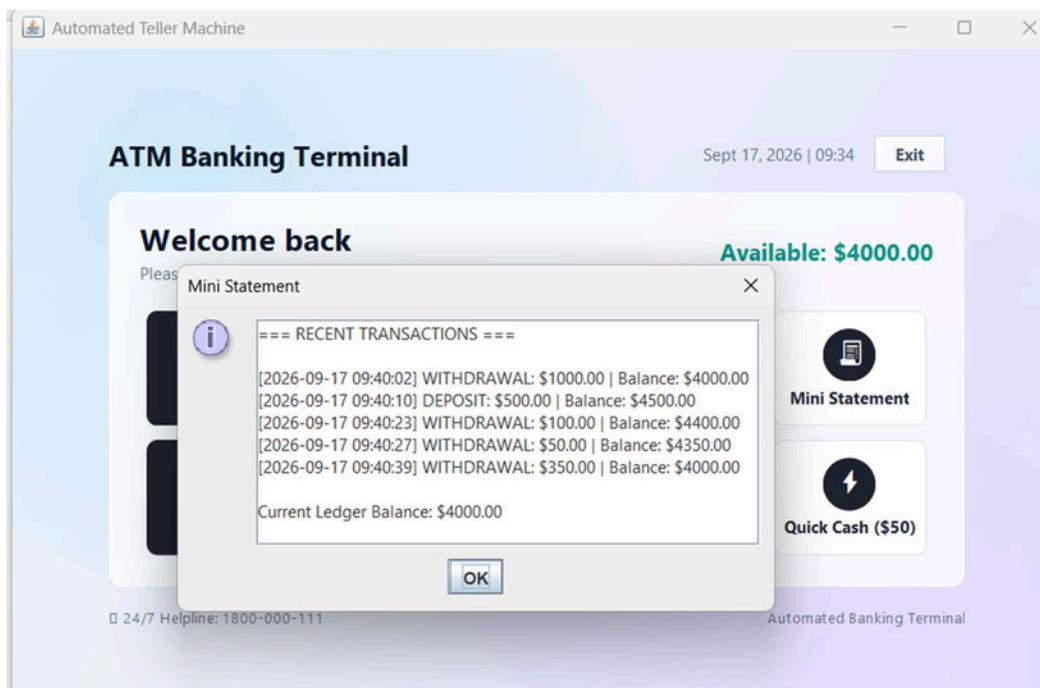


Figure 8: Mini Statement Dialog — Formatted log displaying recent deposits and withdrawals

10. Testing Approach

Test ID	Target Feature	Input / Scenario	Result
TC-01	Valid Login	Correct ID and PIN entered. Terminal loads account details.	Pass
TC-02	Wrong PIN	Correct ID with incorrect PIN. Displays error alert.	Pass
TC-03	Overdraft Check	Withdrawing more than balance. Blocked with warning dialog.	Pass
TC-04	Negative Deposit	Depositing a negative or zero value. Rejected with error message.	Pass
TC-05	Text in Number Field	Entering letters in amount field. Trapped by try-catch cleanly.	Pass
TC-06	PIN Format Check	Entering a 3-digit PIN. Prompted to enter 4 digits.	Pass
TC-07	Quick Cash	Clicking \$50 preset. Balance updates immediately.	Pass
TC-08	Mini Statement	Checking transaction logs after activity.	Pass

11. Challenges Faced

- Separating GUI from calculation logic: It was tempting to update the balance variable directly inside the button click event. Refactoring the code so button clicks only talk to AccountService took extra planning, but it made the project much easier to debug.
- Handling bad input: Users might click OK on an empty prompt or accidentally enter letters. Ensuring every numeric input was trimmed and guarded with try-catch blocks was necessary to prevent crashes.
- UI layout responsiveness: Default Swing forms can look uneven on different screen resolutions. Wrapping components inside centered containers ensured the card maintained its shape cleanly without stretching.

12. Learnings & Key Takeaways

- Layered architecture saves time: Dividing the application into model, service, and UI layers made it easier to find and fix bugs.
- Better understanding of Swing: I learned how to use CardLayout to move between different screens and Graphics2D to create a better-looking interface instead of using the basic grey Swing design.
- Defensive coding: Checking and validating inputs in the service layer helped keep the banking operations safe even if something goes wrong in the UI.
- Version control habits: Making regular commits in Git helped keep the project organized and made it easier to track the development process.

13. Future Enhancements

- Database connectivity: Linking the service layer to SQLite or MySQL via JDBC so accounts and history survive after the app is closed.
- Real recipient account transfers: Updating recipient balances across accounts instead of only deducting from the sender.
- Receipt export: Allowing users to download or print a text file receipt of their transactions.
- Two-Factor Authentication: Simulating a one-time OTP message for large cash transfers.

14. References

1. GitHub Repository: <https://github.com/adityarghh/ApexATM-Vityarthi.git>
2. Oracle Java Documentation: Swing & AWT Architecture (docs.oracle.com/javase/)
3. Oracle Java Tutorials: Creating a GUI With JFC/Swing (docs.oracle.com/javase/tutorial/uiswing/)
4. VITyarthi Project Guidelines: Build Your Own Project Instructions & Submission Guidelines